

SOLID Principles and AI-Assisted Code Refactoring

** Get the required code from resources.*

Section 1: Code Smell Identification

Question 1

List all responsibilities handled by the Bank class.
Which violate the Single Responsibility Principle (SRP)?

Question 2

Identify all usages of if-else and instanceof.
Why do these violate the Open/Closed Principle (OCP)?

Question 3

Identify tightly coupled parts of the system.
Explain how they violate the Dependency Inversion Principle (DIP).

Question 4

Analyze the BankingOperations interface.
Which methods are unnecessary for ATM and MobileApp?
Which principle is violated?

Question 5

Check whether subclasses of Account follow Liskov Substitution Principle (LSP).
Explain your reasoning.

Section 2: Manual Refactoring (Without LLM)

Question 6

Manually refactor the Bank class to follow SRP.
List the new classes and their responsibilities.

Question 7

**Manually remove instanceof usage from withdraw() using polymorphism.
Redesign the Account hierarchy.**

Question 8

**Refactor account creation logic manually using a suitable design pattern.
Explain your design.**

Question 9

**Split the BankingOperations interface manually to follow ISP.
Define the new interfaces.**

Question 10

**Refactor LoanService manually to follow Dependency Inversion Principle.
Show how dependencies are injected.**

Section 3: LLM-Based Refactoring

Question 11

**Write a prompt to refactor the Bank class using SRP.
Use an LLM and generate the refactored code.**

Question 12

**Write a prompt to remove instanceof using polymorphism.
Generate and analyze the output.**

Question 13

**Use an LLM to redesign account creation using Factory Pattern.
Provide the prompt and generated solution.**

Question 14

**Use an LLM to refactor the BankingOperations interface into smaller
interfaces.
Evaluate the response.**

Question 15

Use an LLM to apply Dependency Injection in LoanService.
Show the generated code.

Section 4: Comparison (Manual vs LLM)

Question 16

Compare manual refactoring vs LLM-generated refactoring for SRP.
Which approach produced cleaner design and why?

Question 17

Compare how polymorphism was implemented manually vs by LLM.
Which solution is more extensible?

Question 18

Evaluate the correctness of LLM-generated Factory Pattern implementation.
Did it fully follow OCP?

Question 19

Compare interface segregation done manually vs via LLM.
Which version avoids unnecessary methods better?

Question 20

Compare dependency injection implementations.
Which version is more flexible and testable?

Section 5: Advanced LLM Thinking

Question 21

Did the LLM-generated code introduce any new design issues?
Identify and explain.

Question 22

How many iterations (prompts) were needed to get a good design from the LLM?

What improvements were made in each iteration?

Question 23

What instructions/prompts improved the LLM output the most?

Question 24

**Can LLM fully replace manual design thinking in this case?
Justify your answer.**

Section 6: Reflection

Question 25

Which approach (manual vs LLM) gave you deeper understanding of SOLID principles?

Question 26

In real-world projects, when should you rely on LLM and when should you not?

Question 27

What are the risks of blindly using LLM-generated refactoring?